

```
--
=====
=====
-- ESQUEMA SUPABASE/POSTGRESQL – PROYECTO
TRADER IA
-- Unifica: Freqtrade (motor FreqAI), grid
bots existentes en Binance,
-- señales de IA y alertas, en una sola
base de datos local.
--
=====
=====
--
-- NOTA IMPORTANTE:
-- Freqtrade, cuando se le configura
`db_url` apuntando a este PostgreSQL,
-- gestiona y migra SUS PROPIAS tablas
automáticamente (trades, orders,
-- pairlocks, etc.) vía SQLAlchemy. NO
crear ni modificar esas tablas a
-- mano: dejarlas vivir en su propio
esquema para evitar colisiones con
-- futuras migraciones de Freqtrade.
--
-- Por eso, todo lo de este fichero vive en
el esquema `app`, separado
-- del esquema que use Freqtrade
(normalmente `public`). El frontend
-- (React / React Native) debe leer siempre
desde `app`, nunca consultar
```

```

-- directamente las tablas internas de
Freqtrade.
--
=====
=====

create schema if not exists app;

-- -----
-----
-- 1. strategies – registro unificado de
toda estrategia activa,
-- sea un bot de Freqtrade o uno de los
grid bots manuales existentes.
-- -----
-----

create table if not exists app.strategies (
    id                uuid primary key
default gen_random_uuid(),
    name              text not
null,                -- ej.
'freqtrade_freqai_main', 'grid_sol_usdt'
    source_type       text not null check
(source_type in ('freqtrade', 'grid_bot',
'manual')),
    exchange          text not null default
'binance',
    pair
text,                -- null si
gestiona múltiples pares (ej. Freqtrade
whitelist)
    status            text not null default
'paused' check (status in ('running',
'paused', 'stopped', 'error')),

```

```

        is_paper_trading boolean not null
default true, -- dry-run vs capital real
        config
        jsonb, --
        snapshot de configuración (stoploss, grid
range, etc.)
        created_at      timestampz not null
default now(),
        updated_at      timestampz not null
default now()
    );

comment on table app.strategies is
'Catálogo único de estrategias activas,
independientemente del motor que las
ejecute.';

-- -----
-----
-- 2. unified_trades – operaciones
cerradas/abiertas de CUALQUIER origen.
-- Para Freqtrade, esta tabla se rellena
vía la skill/webhook tras
-- cada evento de entrada/salida (no
leyendo directamente sus tablas
-- internas, que pueden cambiar de
versión a versión).
-- -----
-----
create table if not exists
app.unified_trades (
    id                uuid primary key
default gen_random_uuid(),
    strategy_id       uuid not null

```

```

references app.strategies(id) on delete
cascade,
    exchange_trade_id
text, -- id nativo
en Freqtrade/Binance, si existe
    pair          text not null,
    side          text not null check
(side in ('buy', 'sell')),
    amount        numeric not null,
    price         numeric not null,
    fee           numeric default 0,
    profit_abs
numeric, --
beneficio en cotizado (ej. USDT), solo en
cierres
    profit_pct     numeric,
    opened_at      timestamptz not null,
    closed_at      timestamptz,
    is_paper_trading boolean not null
default true,
    raw_payload
jsonb, --
respuesta cruda del endpoint, por auditoría
    created_at     timestamptz not null
default now()
);

create index if not exists
idx_unified_trades_strategy on
app.unified_trades(strategy_id);
create index if not exists
idx_unified_trades_pair on
app.unified_trades(pair);
create index if not exists

```

```

idx_unified_trades_opened_at on
app.unified_trades(opened_at desc);

-- -----
-----
-- 3. ai_signals – cada señal/decisión
generada por un modelo de IA
--   (FreqAI o el agente conversacional),
CON su justificación.
--   Es la pieza clave contra el riesgo de
"caja negra".
-- -----
-----
create table if not exists app.ai_signals (
    id          uuid primary key
default gen_random_uuid(),
    strategy_id  uuid not null
references app.strategies(id) on delete
cascade,
    pair        text not null,
    signal_type  text not null check
(signal_type in ('buy', 'sell', 'hold',
'force_action')),
    confidence   numeric check
(confidence >= 0 and confidence <= 1),
    model_name
text,          -- ej.
'freqai_lightgbm_v3'
    features
jsonb,         --
variables de entrada usadas para la
decisión
    reasoning
text,          --

```

```

explicación legible (de FreqAI o del LLM)
    executed          boolean not null
default false,      -- si la señal se llegó a
ejecutar como operación
    trade_id          uuid references
app.unified_trades(id),
    created_at        timestampz not null
default now()
);

create index if not exists
idx_ai_signals_strategy on
app.ai_signals(strategy_id);
create index if not exists
idx_ai_signals_created_at on
app.ai_signals(created_at desc);

-- -----
-----
-- 4. portfolio_snapshots – fotos
periódicas del balance, para graficar
-- evolución del patrimonio en el
dashboard de React.
-- -----
-----
create table if not exists
app.portfolio_snapshots (
    id                uuid primary key
default gen_random_uuid(),
    strategy_id        uuid references
app.strategies(id) on delete cascade, --
null = balance global
    currency           text not null default
'USDT',

```

```

        balance_free    numeric not null,
        balance_used    numeric not null
default 0,
        balance_total  numeric not null,
        snapshot_at    timestampz not null
default now()
);

create index if not exists
idx_snapshots_strategy_time on
app.portfolio_snapshots(strategy_id,
snapshot_at desc);

-- -----
--
-- 5. exchange_credentials – claves de API,
SIEMPRE cifradas en origen.
--     Nunca se exponen al frontend; solo el
backend/skill las lee.
-- -----
--
create table if not exists
app.exchange_credentials (
    id                uuid primary key
default gen_random_uuid(),
    exchange          text not null,
    label             text not
null,                -- ej.
'binance_main'
    api_key_encrypted text not null,
    api_secret_encrypted text not null,
    created_at        timestampz not null
default now()
);

```

```

-- -----
-----
-- 6. alerts – eventos que deben
-- notificarse (vía skill / push / Telegram)
-- -----
-----
create table if not exists app.alerts (
    id            uuid primary key
default gen_random_uuid(),
    strategy_id   uuid references
app.strategies(id) on delete cascade,
    level        text not null check
(level in ('info', 'warning', 'critical')),
    message      text not null,
    acknowledged boolean not null
default false,
    created_at   timestampz not null
default now()
);

create index if not exists
idx_alerts_unacknowledged on
app.alerts(acknowledged) where acknowledged
= false;

-- -----
-----
-- Trigger genérico para mantener
-- updated_at en strategies
-- -----
-----
create or replace function
app.set_updated_at()

```



```

returns trigger as $$
begin
    new.updated_at = now();
    return new;
end;
$$ language plpgsql;

drop trigger if exists
trg_strategies_updated_at on
app.strategies;
create trigger trg_strategies_updated_at
    before update on app.strategies
    for each row execute function
app.set_updated_at();

--
=====
=====
-- ROW LEVEL SECURITY (Supabase)
-- Uso personal: una sola cuenta de
usuario, pero se habilita RLS de
-- todas formas como buena práctica,
restringiendo a usuarios autenticados.
--
=====
=====
alter table app.strategies enable row level
security;
alter table app.unified_trades enable row
level security;
alter table app.ai_signals enable row level
security;
alter table app.portfolio_snapshots enable
row level security;

```

```
alter table app.alerts enable row level
security;
-- exchange_credentials NO se expone vía
RLS a clientes; solo accede el
-- backend con la service_role key, por lo
que deliberadamente no se
-- define política de lectura para
anon/authenticated.

create policy
"authenticated_read_strategies" on
app.strategies
    for select using (auth.role() =
'authenticated');
create policy "authenticated_read_trades"
on app.unified_trades
    for select using (auth.role() =
'authenticated');
create policy "authenticated_read_signals"
on app.ai_signals
    for select using (auth.role() =
'authenticated');
create policy
"authenticated_read_snapshots" on
app.portfolio_snapshots
    for select using (auth.role() =
'authenticated');
create policy "authenticated_read_alerts"
on app.alerts
    for select using (auth.role() =
'authenticated');

--
=====
```

```
=====
-- Vista de conveniencia para el dashboard:
últimas operaciones + señal
-- de IA que las originó (si la hay).
--
=====
=====
create or replace view
app.v_trades_with_signal as
select
    t.*,
    s.signal_type,
    s.confidence,
    s.model_name,
    s.reasoning
from app.unified_trades t
left join app.ai_signals s on s.trade_id =
t.id
order by t.opened_at desc;
```